

Sistema de Gestão de Veículos (CRUDCarro)

Documentação Técnica: Arquitetura MVC, JavaFX com FXML, Princípios SOLID e Fluxo de Execução

1. Visão Geral e Arquitetura do Projeto

A aplicação é estruturada sobre o padrão **MVC (Model-View-Controller)** desacoplado, reforçado pelas boas práticas da Orientação a Objetos e princípios **SOLID**. O sistema divide suas responsabilidades em camadas estritas:

Pacote / Camada	Classes Principais	Responsabilidade no Sistema
main	Main.java	Ponto de entrada; instancia dependências e configura a Fábrica de Controladores.
controller	MainController.java	Gerencia exclusivamente os eventos da tela FXML (cliques, inputs, binding, confirmações visuais).
service	ICarroService CarroService.java	Regras de negócio, validações no backend (ICarroValidador), montagem de DTO e persistência.
validator	Validador<T>, ICarroValidador CarroValidador, etc.	Cadeia polimórfica de validação (campos obrigatórios, ano e modelo). Lança exceções de negócio.
model.dao	ICarroDAO CarroDAO.java	Operações SQL diretas no PostgreSQL, com registro de falhas via Logger.
model.dto	CarroDTO.java	Objeto puro de transferência de dados (POJO com getters e setters).
util	DialogUtil.java	Padronização de diálogos visuais (Sucesso, Aviso, Erro e Confirmação).

2. Ciclo de Inicialização: O que acontece ao rodar o sistema?

- 1. Instanciação Externa:** No método `start()` da classe `Main`, são instanciados `CarroDAO` e `CarroValidador`, sendo ambos injetados no `CarroService`.
- 2. Fábrica de Controladores:** É configurado `loader.setControllerFactory(...)`. Dessa forma, quando o `FXMLLoader` lê o FXML, ele solicita à fábrica a instância do `MainController` injetando apenas o `ICarroService` via construtor (DIP).
- 3. Inicialização dos Componentes (initialize):** O controller conecta as colunas da `TableView` às propriedades do `CarroDTO`, configura os listeners de seleção e o filtro dinâmico de busca, carregando os registros do banco na tela.

3. Fluxo de Execução Passo a Passo (O que acontece ao Clicar?)

A. Clicar no botão 'Adicionar' (Cadastrar Carro)

- **Captura e Delegação:** O método `btnAdicionarAction` obtém os textos dos campos e chama `carroService.cadastrar(...)`.
- **Validação no Backend:** O `CarroService` aciona o `CarroValidador`, que percorre com `foreach` os validadores (`CamposObrigatoriosValidador`, `ModeloValidador` e `AnoValidador`). Se algum falhar, o backend lança `IllegalArgumentException` e a tela exibe um aviso com `DialogUtil.showWarning`.
- **Persistência:** Aprovado na validação, o serviço cria o `CarroDTO` e chama `carroDAO.inserirCarro(carro)`.
- **Retorno Visual:** Exibe diálogo de sucesso (`DialogUtil.showInfo`), reseta o formulário e atualiza a tabela.

B. Clicar em uma linha da Tabela

- O evento `carregarCampos()` resgata o item selecionado e preenche os campos de texto.
- Um `ChangeListener` no modelo de seleção da tabela detecta a linha ativa e **habilita os botões Editar e Excluir**, desabilitando o botão Adicionar.

C. Clicar no botão 'Editar' (Atualizar Carro)

- Abre caixa de diálogo de confirmação visual via `DialogUtil.showConfirmation(...)`.
- Se confirmado, chama `carroService.atualizar(...)`, que valida o ID e os novos dados no backend e executa o SQL `UPDATE`.
- Notifica sucesso, limpa os campos e recarrega os dados na tela.

D. Clicar no botão 'Excluir'

- Confirma a intenção com o usuário via `DialogUtil.showConfirmation(...)`.
- Se confirmado, chama `carroService.excluir(idStr)`, que valida o ID no backend e executa o comando SQL `DELETE` no DAO.
- Notifica com `DialogUtil.showInfo` e atualiza a lista de veículos.

E. Digitar na Barra de Pesquisa

- A cada caractere digitado, o listener do `txtPesquisa` atualiza o predicado de uma `FilteredList`.
- O método `carroService.correspondeATermo(carro, termo)` testa se o texto bate com `Marca`, `Modelo`, `Placa` ou `ID` em tempo real, sem novas consultas ao banco.

4. Aplicação e Importância dos Princípios SOLID

S — Single Responsibility Principle (Responsabilidade Única)

Cada classe possui apenas uma única razão para mudar. O `Controller` cuida exclusivamente da tela e eventos FXML de UI; o `Service` orquestra as regras de negócio e validações; o `DAO` executa persistência SQL; os validadores checam conformidade dos dados e o `DialogUtil` cuida de popups.

O — Open/Closed Principle (Aberto/Fechado)

A arquitetura está aberta para extensão e fechada para modificação. Ao criar uma nova validação (ex.: formato de `Placa Mercosul`), cria-se uma nova classe implementando `Validador<String>` e a insere na lista, sem alterar nenhuma linha dos validadores já testados.

L — Liskov Substitution Principle (Substituição de Liskov)

Todas as implementações de `Validador<T>` podem ser substituídas transparentemente pela abstração base sem quebrar a execução ou gerar exceções inesperadas dentro da lista genérica percorrida pelo `foreach`.

I — Interface Segregation Principle (Segregação de Interfaces)

Interfaces pequenas e especializadas (`Validador<T>`, `ICarroValidador`, `ICarroService`, `ICarroDAO`) em vez de interfaces monolíticas. Nenhuma classe é obrigada a implementar métodos irrelevantes para sua função.

D — Dependency Inversion Principle (Inversão de Dependência)

Módulos de alto nível não dependem de módulos de baixo nível. O `Controller` depende unicamente de `ICarroService`; o `CarroService` depende de `ICarroDAO` e `ICarroValidador`. A injeção ocorre via construtor na `Fábrica de Controladores` do `Main`.

5. Tratamento de Erros e Boas Práticas Adicionais

- **Logger no DAO:** Conforme exigido nos slides, o `CarroDAO` utiliza `java.util.logging.Logger` para registrar a stack trace técnica com `logger.log(Level.SEVERE, ..., e)` sem expor detalhes sensíveis ao usuário final.
- **Ausência de Console para Alertas:** Toda interação com o operador é feita via `DialogUtil` (Alerts do JavaFX).
- **Padrões de Nomenclatura:** Prefixos canônicos JavaFX (`btn`, `txt`, `lbl`, `tbl`, `col`) e métodos semânticos (`btnAdicionarAction`, `btnEditarAction`).

6. Roteiro para Apresentação e Defesa Oral

1. **Demonstração Prática:** Cadastre um veículo válido, mostre o alerta de sucesso e a inserção na tabela.
2. **Demonstração de Validação no Backend:** Tente submeter com campos vazios ou modelo com dígitos; mostre que a validação é disparada pelo backend (`Service`) e o alerta é exibido.
3. **Destaque do OCP:** Abra `CarroValidador.java` e aponte para a lista polimórfica e o loop `foreach`.
4. **Destaque do SRP e DIP:** Abra `MainController.java` (apenas FXML e UI) e `CarroService.java` (regras e validações), e mostre `Main.java` com `setControllerFactory`.
5. **Destaque do Tratamento de Erros:** Abra `CarroDAO.java` mostrando o `Logger` em cada bloco `catch`.